

12.12 Beacons Manual

12.13 Introducción

BeaconList es una lista enlazada simple que implementa la **Tabla de Nodos Vecinos** (*Neighbour Node Table*, NNT) utilizada en protocolos de enrutamiento vehicular dentro del simulador **Veins** (OMNeT++ + SUMO).

Cada vehículo o RSU (*Road-Side Unit*) mantiene una instancia de **BeaconList**. Cuando un nodo recibe un *beacon* periódico de un vecino, almacena o actualiza en esta tabla las métricas de enrutamiento incluidas en ese mensaje: posición, distancias relativas, densidad local, ancho de banda disponible e información del siguiente salto (*next-hop*) seleccionado por el vecino.

12.13.1 Archivos fuente

- `beacon.h` 2014 declaración de la clase y documentación de la interfaz.
- `beacon.cc` 2014 implementación de todos los métodos.

Ambos archivos residen en:

`src/veins/modules/application/traci/`

12.13.2 Estructura interna de un entrada (beacon)

Cada entrada de la NNT es un `struct beacon` con los siguientes campos:

Campo	Tipo	Descripción
<code>typeNode</code>	<code>std::string</code>	Tipo de nodo: "rsu" o identificador de vehículo
<code>idMsg</code>	<code>int</code>	ID del mensaje/árbol del último beacon recibido
<code>idVehicle</code>	<code>int</code>	ID MAC del vecino (clave primaria de la tabla)
<code>time</code>	<code>double</code>	Tiempo de simulación [s] en que se recibió el beacon
<code>SenderCoord</code>	<code>veins::Coord</code>	Posición (x, y) del vecino en el instante del beacon
<code>dsrc</code>	<code>double</code>	Dist. Fuente2013Relay [m]: del origen del paquete al vecino
<code>dsd</code>	<code>double</code>	Dist. Emisor2013Destino [m]: del vecino al RSU destino
<code>drd</code>	<code>double</code>	Dist. Relay2013Destino [m]: del nodo local al destino
<code>Nv</code>	<code>double</code>	Densidad de vehículos reportada por el vecino (normalizada)
<code>wdist</code>	<code>double</code>	Puntuación de distancia ponderada (usada en ordenación)
<code>abe</code>	<code>double</code>	Ancho de banda disponible estimado [bps]
<code>NH_Address</code>	<code>LAddress::L2Type</code>	Dirección L2 del siguiente salto seleccionado por el vecino
<code>NH_GS</code>	<code>double</code>	Puntuación global (<i>global score</i>) del next-hop del vecino
<code>NH_Dst_to_Dest</code>	<code>double</code>	Dist. [m] del next-hop del vecino al destino

Cuadro 12.5: Campos del `struct beacon` (entrada de la NNT).

12.14 Ciclo de vida típico en una capa de aplicación

El siguiente fragmento muestra el patrón estándar de uso de **BeaconList** dentro de un método **onWSM()** o **onBSM()** de Veins:

```

1 // Declaracion en la clase de la aplicacion (p.ej. MyVeinsApp.h)
2 BeaconList nnt;
3
4 // Al recibir un beacon de un vecino:
5 void MyApp::onWSM(BaseFrame1609_4* frame) {
6     int senderId    = frame->getSenderAddress();
7     double now      = simTime().dbl();
8     Coord sCoord    = ...; // posicion del emisor extraida del frame
9
10    // 1. Eliminar entradas caducadas (TTL = 2 segundos)
11    nnt.PurgeBeacons(2.0);
12
13    // 2. Agregar o actualizar la entrada del vecino
14    if (!nnt.SearchBeaconNodeID(senderId)) {
15        nnt.AddBeacon("vehicle", msgId, senderId, now, sCoord,
16                      dsr, dsd, drd, Nv, wdist, abe,
17                      nhAddr, nhGS, nhDst);
18    } else {
19        nnt.UpdateBeacon("vehicle", msgId, senderId, now, sCoord,
20                          dsr, dsd, drd, Nv, wdist, abe,
21                          nhAddr, nhGS, nhDst);
22    }
23
24    // 3. Ordenar para que TopTable() devuelva el mejor candidato
25    nnt.SortBeacons();
26
27    // 4. Seleccionar el mejor vecino para el reenvio
28    int nextHop = nnt.TopTable();
29 }

```

Listing 12.7: Patrón de uso general de BeaconList.

12.15 Referencia de funciones

12.15.1 Constructor y destructor

BeaconList()

Firma:

```
1 BeaconList();
```

Descripción: Inicializa una tabla vacía. Los punteros internos `head`, `curr` y `temp` se establecen en `nullptr`.

Ejemplo:

```
1 BeaconList nnt; // tabla lista para usar
```

~BeaconList()

Firma:

```
1 ~BeaconList();
```

Descripción: Recorre la lista y libera con `delete` cada nodo asignado dinámicamente, evitando fugas de memoria al destruir el objeto. Se invoca automáticamente cuando la variable sale de scope o el módulo OMNeT++ es eliminado.

12.15.2 Mutadores

AddBeacon()

Firma:

```
1 void AddBeacon(std::string b_typeNode, int b_idMsg, int b_idVehicle,
2               double b_time, veins::Coord b_SenderCoord,
3               double b_dsr, double b_dsd, double b_drd,
4               double b_Nv, double b_wdist, double b_abe,
5               veins::LAddress::L2Type b_NH_Address,
6               double b_NH_GS, double b_NH_Dst_to_Dest);
```

Descripción: Crea un nuevo nodo con los valores suministrados y lo anexa al final de la lista. Debe llamarse únicamente cuando `SearchBeaconNodeID()` devuelve `false` para el ID del emisor, es decir, cuando el vecino aún no existe en la tabla.

Parámetros relevantes:

- `b_typeNode` 2014 "rsu" para unidades de infraestructura, cualquier otra cadena para vehículos.
- `b_idVehicle` 2014 identificador único del vecino; es la clave primaria para búsquedas y actualizaciones.
- `b_time` 2014 valor de `simTime().dbl()` en el momento de la recepción; usado por `PurgeBeacons()` para calcular expiración.
- `b_wdist`, `b_Nv` 2014 métricas que `SortBeacons()` usa para calcular la puntuación compuesta.

Ejemplo:

```
1 nnt.AddBeacon(
2   "vehicle",           // tipo de nodo
3   42,                  // ID del mensaje/arbol
4   17,                  // ID MAC del vecino
5   simTime().dbl(),     // timestamp actual
```

```

6      mobility->getPositionAt(simTime()),
7      150.0,                      // dsr [m]
8      300.0,                      // dsd [m]
9      280.0,                      // drd [m]
10     0.6,                        // Nv (densidad normalizada)
11     0.75,                      // wdist (puntuacion distancia)
12     6e6,                       // abe [bps]
13     5,                         // NH_Address
14     0.8,                       // NH_GS
15     120.0                      // NH_Dst_to_Dest [m]
16 );

```

UpdateBeacon()

Firma: igual que AddBeacon(), con el mismo conjunto de parámetros.

Descripción: Localiza la entrada cuyo `idVehicle` coincide con `b_idVehicle` y sobrescribe todos sus campos con los nuevos valores. Si el ID no existe en la tabla, la llamada es un *no-op*. Debe llamarse cuando `SearchBeaconNodeID()` devuelve `true`.

Ejemplo:

```

1  if (nnt.SearchBeaconNodeID(senderId)) {
2      nnt.UpdateBeacon("vehicle", newMsgId, senderId,
3                      simTime().dbl(), newCoord,
4                      dsr, dsd, drd, Nv, wdist, abe,
5                      nhAddr, nhGS, nhDst);
6  }

```

DeleteBeacon()

Firma:

```

1  void DeleteBeacon(int b_idVehicle);

```

Descripción: Elimina de la lista la entrada cuyo `idVehicle` coincide con el parámetro y libera la memoria con `delete`. Maneja correctamente el caso en que la entrada a eliminar sea la cabeza de la lista. Si el ID no existe, la llamada es un *no-op*.

Ejemplo:

```

1  // Eliminar manualmente al vecino con ID 17
2  nnt.DeleteBeacon(17);

```

PurgeBeacons()

Firma:

```

1  void PurgeBeacons(double b_ttl);

```

Descripción: Recorre la tabla completa y elimina todas las entradas cuyo campo `time` sea anterior a `(simTime() - b_ttl)`. Debe invocarse al inicio de cada recepción de

beacon para mantener la tabla actualizada. Usa **delete** (no **free**) para ser consistente con la alocaión dinámica de C++.

Parámetros:

- `b_ttl` 2014 umbral de vida útil en segundos. Valor típico en VANETs: **2.0 s**.

Ejemplo:

```
1 // Al inicio del handler de recepcion, depurar entradas antiguas
2 nnt.PurgeBeacons(2.0);
```

SortBeacons()

Firma:

```
1 void SortBeacons();
```

Descripción: Ordena la lista en orden **descendente** según la puntuación compuesta:

$$GS = \frac{wdist + N_v}{2}$$

Tras la ordenación, el nodo en la cabeza de la lista es el mejor candidato para el reenvío del paquete. Utiliza *bubble sort*; esto es adecuado dado que las tablas VANET raramente superan los 30 vecinos simultáneos.

Ejemplo:

```
1 nnt.SortBeacons();
2 int mejorVecino = nnt.TopTable(); // cabeza de la lista ordenada
```

12.15.3 Accesores del primer elemento (Top-of-Table)

Todos los accesores de esta sección leen únicamente el primer nodo de la lista. Deben invocarse **después** de `SortBeacons()` para que el nodo en la cabeza sea el candidato óptimo. Devuelven `0`, `0.0` o un `Coord()` por defecto si la tabla está vacía.

TopTable()

```
1 int TopTable();
```

Devuelve el `idVehicle` (ID MAC) del mejor candidato al reenvío. Este valor se usa como dirección destino del paquete a reenviar.

```
1 nnt.SortBeacons();
2 int nextHopId = nnt.TopTable(); // 0 si la tabla esta vacia
3 sendPacket(nextHopId, payload);
```

TopTableNv()

```
1 double TopTableNv();
```

Devuelve la densidad local N_v del mejor candidato. Útil para algoritmos que toman decisiones basadas en la congestión de la red.

```
1 double density = nnt.TopTableNv();
2 if (density > DENSITY_THRESHOLD) { /* alta congestión */ }
```

TopTableNghCoord()

```
1 veins::Coord TopTableNghCoord();
```

Devuelve la posición (x, y) del mejor vecino candidato al reenvío.

```
1 Coord nhPos = nnt.TopTableNghCoord();
2 double d = myPos.distance(nhPos);
```

TopTable_U_dist()

```
1 double TopTable_U_dist();
```

Devuelve el campo **wdist** (puntuación de distancia ponderada) del mejor candidato. Indica qué tan favorable es la posición geográfica del vecino respecto al destino.

TopTabledisToDestination()

```
1 double TopTabledisToDestination();
```

Devuelve **dsd**: la distancia en metros desde el mejor vecino hasta el destino (RSU). Un valor menor indica que el vecino está más cerca del destino.

```
1 double distToRSU = nnt.TopTabledisToDestination();
```

topTableDtoS()

```
1 double topTableDtoS();
```

Devuelve **dsr**: la distancia en metros desde el nodo **origen** del paquete hasta el mejor vecino (Dist. Fuente2013Relay).

TopTableGS()

```
1 double TopTableGS();
```

Devuelve **NH_GS**: la puntuación global del siguiente salto que el propio mejor vecino ha seleccionado. Permite evaluar la calidad del enlace de dos saltos.

```
1 double twoHopScore = nnt.TopTableGS();
```

12.15.4 Utilidades de búsqueda

CounterBeacons()

```
1 int CounterBeacons();
```

Descripción: Recorre toda la lista y devuelve el número de entradas activas. Se usa habitualmente para calcular la densidad local de vehículos que el nodo propio reportará en sus beacons periódicos.

```
1 int Nv = nnt.CounterBeacons();  
2 // Enviar Nv en el siguiente beacon propio  
3 beacon->setNv((double)Nv / MAX_NEIGHBORS);
```

SearchBeaconNodeID()

```
1 bool SearchBeaconNodeID(int NodeID);
```

Descripción: Devuelve **true** si existe una entrada con **idVehicle == NodeID**. Es el **selector** que determina si debe llamarse **AddBeacon()** o **UpdateBeacon()** tras recibir un beacon.

```
1 if (!nnt.SearchBeaconNodeID(senderId))  
2     nnt.AddBeacon(...);  
3 else  
4     nnt.UpdateBeacon(...);
```

SearchBeaconTreeID()

```
1 bool SearchBeaconTreeID(int treeID);
```

Descripción: Devuelve **true** si alguna entrada tiene **idMsg == treeID**. Útil para protocolos que propagan mensajes de árbol y necesitan detectar duplicados.

```
1 if (nnt.SearchBeaconTreeID(incomingMsgId)) {  
2     // mensaje ya conocido, descartar  
3     return;  
4 }
```

SearchBeaconCoord()

```
1 veins::Coord SearchBeaconCoord(int NodeID);
```

Descripción: Busca el nodo con **idVehicle == NodeID** y devuelve su campo **SenderCoord**. Si el ID no existe, devuelve un **Coord()** construido por defecto.

```
1 Coord pos = nnt.SearchBeaconCoord(targetId);  
2 double dist = myCoord.distance(pos);
```

SearchClstNODE()

```
1 int SearchClstNODE(veins::Coord DstNodeCoord);
```

Descripción: Busca entre todos los vecinos de tipo vehículo (excluye RSUs) aquel cuya posición es más cercana a **DstNodeCoord**. Devuelve su **idVehicle**, o **0** si ningún vehículo vecino se encuentra dentro del umbral de 4000 m.

Se usa en estrategias de enrutamiento geográfico (*greedy forwarding*) cuando la tabla ya fue depurada y ordenada.

```
1 Coord rsuPosition = getRSUCoord();
2 int closestId = nnt.SearchClstNODE(rsuPosition);
3 if (closestId != 0)
4     forwardTo(closestId, packet);
```

SearchBeaconRSU()

```
1 int SearchBeaconRSU();
```

Descripción: Recorre la tabla y devuelve el **idVehicle** del primer RSU encontrado (entrada con **typeNode == "rsu"**). Devuelve **0** si no hay ningún RSU en la NNT.

```
1 int rsuId = nnt.SearchBeaconRSU();
2 if (rsuId != 0) {
3     // entregar paquete directamente al RSU
4     sendDirect(rsuId, packet);
5 }
```

Search_RSU_NNT()

```
1 bool Search_RSU_NNT();
```

Descripción: Versión booleana de **SearchBeaconRSU()**. Devuelve **true** en cuanto encuentra al menos un RSU en la tabla. Útil como guarda en lógica condicional sin necesitar el ID del RSU.

```
1 if (nnt.Search_RSU_NNT()) {
2     // hay cobertura de infraestructura directa
3     mode = INFRASTRUCTURE_MODE;
4 } else {
5     // solo vecinos vehiculares, usar multisalto
6     mode = AD_HOC_MODE;
7 }
```

12.15.5 Depuración

PrintBeacons()

```
1 void PrintBeacons(int CurrNodeAddress, std::string IN_OUT);
```


Descripción: Imprime el contenido completo de la NNT en el log de OMNeT++ mediante el stream EV. La salida incluye una cabecera formateada con el ID del nodo local y la etiqueta IN_OUT (p.ej. "IN" al recibir, "OUT" al reenviar), seguida de una fila por cada entrada de la tabla.

Solo se ejecuta cuando el nivel de log de la simulación lo permite (EV es silencioso en ejecuciones sin interfaz gráfica por defecto).

```
1 // Imprimir la tabla antes del reenvio
2 nnt.PrintBeacons(myAddress, "OUT");
```

Salida de ejemplo en el log de OMNeT++:

```
-----
OUT --> Vehicle Neighbour Table CurrNodeAddress=3
-----
NodeType TreeMsg_ID MAC_Address NH_Address NH_GS ...
vehicle      42      17          5    0.8 ...
rsu          7       2          0    1.0 ...
-----
```

12.16 Flujo de decisión de reenvío

La figura ?? resume el flujo completo desde la recepción de un beacon hasta la selección del próximo salto.

1. **Recepción** del beacon de un vecino v .
2. **PurgeBeacons(TTL)** 2014 eliminar entradas caducadas.
3. **SearchBeaconNodeID(v)**:
 - `false` $\Rightarrow$ **AddBeacon()** 2014 primer beacon de v .
 - `true` $\Rightarrow$ **UpdateBeacon()** 2014 beacon periódico de v .
4. **SortBeacons()** 2014 ordenar por $GS = (wdist + N_v)/2$.
5. Decisión de reenvío:
 - **Search_RSU_NNT()** = `true` $\Rightarrow$ reenviar directamente al RSU (**SearchBeaconRSU()**).
 - Si no, usar **TopTable()** como next-hop vehicular.
 - Como alternativa geográfica: **SearchClstNODE(rsuCoord)**.

12.17 Integración con la capa de aplicación de Veins

Para integrar **BeaconList** en una aplicación Veins derivada de **DemoBaseApplLayer**:

1. Incluir el header en la cabecera de la aplicación:

```
1 #include "veins/modules/application/traci/beacon.h"
```

2. Declarar una instancia miembro en la clase:

```
1 class MyApp : public DemoBaseApplLayer {
2     BeaconList nnt; // tabla de vecinos de este nodo
3     ...
4 };
```

3. Procesar los beacons entrantes en `onWSM()` o `onBSM()`:

```
1 void MyApp::onWSM(BaseFrame1609_4* frame) {
2     nnt.PurgeBeacons(2.0);
3
4     int sid = frame->getSenderAddress();
5     if (!nnt.SearchBeaconNodeID(sid))
6         nnt.AddBeacon(/* campos del frame */);
7     else
8         nnt.UpdateBeacon(/* campos del frame */);
9
10    nnt.SortBeacons();
11 }
```

4. Calcular la densidad propia con `CounterBeacons()` y anunciarla en el próximo beacon:

```
1 void MyApp::handlePositionUpdate(cObject* obj) {
2     int Nv = nnt.CounterBeacons();
3     myBeacon->setNv((double)Nv / NORM_FACTOR);
4     sendDown(myBeacon);
5 }
```

12.18 Consideraciones de rendimiento y limitaciones

- **Complejidad temporal:** todas las operaciones de búsqueda, inserción y eliminación son $O(n)$ donde n es el número de vecinos. `SortBeacons()` tiene complejidad $O(n^2)$ en el peor caso (*bubble sort*). Para redes con alta densidad ($n > 50$) puede ser necesario reemplazarlo por un algoritmo más eficiente.
- **No es hilo-seguro:** los punteros internos `curr` y `temp` son compartidos entre métodos. Nunca llamar a dos métodos simultáneamente sobre la misma instancia desde distintos hilos o eventos OMNeT++ no secuenciales.
- **TTL y frecuencia de beacon:** el valor de `b_ttl` pasado a `PurgeBeacons()` debe ser mayor o igual al intervalo de beacon del protocolo para evitar eliminar vecinos activos. Con beacons cada 1 s, un TTL de 2 s es el mínimo razonable.

- **Umbral de SearchC1stNODE():** el radio máximo de búsqueda está fijado en 4000 m. Si el escenario de simulación supera esa distancia entre nodos debe ajustarse directamente en el código fuente.